[SVT Lab] Final Report: Testing Laboratory Concepts

Reza Ahmadi (VR510067)

March 18, 2026

Contents

1 Introduction

The main purpose of this report is a quick review of all the important topics in the testing part of the Systems Verification and Testing course.

1.1 Verification vs Testing

n Verification, the main approaches are defined to check the design and the designer's intention, and how a design respects that intention. However, in Testing, we check every device that is made; for instance, we check whether the output of a circuit respects our input or not. In short, we do the verification for the final product just one time after all feature implementation, but we do the testing for every circuit or device that is made from the design. Another difference between Verification and Testing is related to the way they are applied; in fact, for Verification, we use simulators to mimic design behavior, but in Testing, we apply it to every single real circuit or device.

1.2 The cost of Faults

One of the most important and notable purposes of testing is to reduce the cost of faults. In fact, if we consider an IC, finding a fault at a lower level (wafer level) is simpler and cheaper than when it is used in a smartphone or a car.

2 Hardware Description Languages: Verilog-AMS

Verilog-AMS (Analog and Mixed-Signal) is an HDL (Hardware Description Language) extension that helps us to combine event-driven simulation (digital) with continuous-time simulation (analog) into a single standard. Verilog-AMS combines two languages (Verilog and Verilog-A) and provides an extension to allow mixed-signal components.

2.1 Mixed-Signal Modeling

Verilog-AMS (Analog and Mixed-Signal) is important because it allows us to simulate complex behavioral systems where digital control logic is connected with analog hardware within a single, unique language. We use mixed modeling for purposes such as:

- **System Integration:** Modern designs, like smart hybrid power car systems, or automatic braking, need a digital controller to control analog components like motors, fuel tanks, or sensors.
- **Domain Convergence:** Mixed modeling helps us to connect the digital world with the analog world.
- **Language combiantion:** Verilog-AMS combines Verilog-HDL and Verilog-A.
- **Simulation Balance:** Mixed Modeling provides a balance between simulation performance (considerd by speed) and accuracy, and we can contrill it between high-accuracy SPICE and high-performance for digital models.

Gemini said Mixed models are demanded for component modeling, especially when we have hardware with several levels of implementation. They also help us to improve the acceleration of the simulation process. Furthermore, they support a top-down design process that helps engineers move from abstract architectural models to detailed implementations.

2.2 Key Concepts

- **Mixed-Signal Nature:** It is about Verilog-AMS, which is a combination of Verilog-HDL and Verilog-A. In fact, Verilog-AMS is a bridge between the digital world and the analog world inside a single module.
- **Disciplines and Natures:** Verilog-AMS supports disciplines such as thermal, mechanical, and rotational, allowing simulation of physical processes other than electrical and electronic. Each discipline is related to a nature that defines some physical characteristics, like potential (for instance, voltage) and flow (for current).
- **Conservative Behavioral Modeling:** Verilog-AMS uses branch contributions (represented by the ¡+ operator) to define relationships between nodes. For example, a resistor is modeled by relating voltage and current: $V(p,n) < +r * I(p,n)$. This allows the analog solver to apply Kirchhoff's laws to solve for unknown voltages and currents.
- **Analog Events and Synchronization:** Verilog-AMS introduces special sensitivity triggers, such as @cross and timer, to synchronize digital and analog domains
- **Accuracy vs. Performance Balance:** It is related to having a balance or trade-off between simulation performance and accuracy.

3 Fault Modeling in Integrated Circuits

Fault modeling is a concept that we use for injecting intended faults into a circuit or system, such as a digital signal being "stuck-at" a specific value like 0 or 1 inside of a chip design. Thanks to this technique, we can test if the circuit's safety mechanisms will successfully detect and handle real-world failures before the physical chip is ever manufactured.

3.1 Digital Fault Models

In digital systems that process discrete data, digital fault models focus on logic errors. The most repeated of this kind of fault is the Stuck-At fault, where a specific node in the circuit is permanently stuck at a digit '0' or '1'. Under standards like ISO 26262, these kinds of errors are categorized by interacting with the chip's safety mechanisms, such as:

- **Single Point Faults:** A fault that directly breaks a system's safety goal because it is not protected by any safety mechanism.
- **Residual Faults:** A fault that breaks a safety goal, meanwhile it is technically covered by a safety mechanism, but the mechanism fails to detect it.
- **Multi-Point Faults:** Faults that only violate a safety mechanism when another fault has occurred and it is combined with that.
- **Latent Multi-Point Faults:** A fault occurring inside the safety mechanism and that goes completely undetected by any other safety mechanisms.
- **Detected Multi-Point Faults:** A fault inside a safety mechanism that is successfully detected by a secondary safety monitor.

3.2 Analog Fault Models

Unlike digital circuits, analog circuits deal with continuous signals. A fault in analog circuits is not just a simple 0 or 1; it can be a kind of shift in a waveform. Analog fault models are categorized into two categories:

- **Catastrophic (Hard) Faults:** These faults represent a huge physical damage in the circuit, modeled as either a complete short circuit or an open circuit.
- **Parametric (Soft) Faults:** These faults represent problems in productions. The circuit doesn't completely fail, but a part of that or a component drifts outside of its acceptable tolerance range.

4 Automatic Test Pattern Generation (ATPG)

ATPG is a kind of EDA method for finding a specific input or test sequence. When we use it on a digital circuit, it helps us to have automatic test equipment to distinguish between the correct circuit behavior and the faulty circuit behavior because of defects. On the other hand, it makes patterns for us to use to test devices after they are manufactured.

4.1 Test Classification

During the test generation process, ATPG classifies faults into several different categories for evaluating the system, such as:

- **Full (FU):** All of the injected faults inside the design.
- **Detected (DT):** Faults successfully detected by the tool, which will be divided into detected by simulation (DS) and detected by implication (DI).
- **Aborted:** Failures where the tool interrupted the generation of tests because it reached the backtrack limit, leaving them undetected.
- **ATPG Untestable (AU):** Failures for which the test generator cannot find test patterns, but is also unable to prove their redundancy.
- **Redundant (RE):** Redundant failures which the test generator considers impossible to detect.
- **Undetected (UD):** Failures the test generator could not detect and failed to prove untestable, which includes Uncontrolled (UC) and Unobserved (UO) faults.

4.2 Coverage Metrics

For evaluating the ATPG process, we can measure it by using three primary metrics:

- **Test Coverage:** It refers to the percentage of faults detected in the middle of all testable faults.
- **Fault Coverage:** It refers to the percentage of faults detected int the middle of all injected faults.
- **ATPG Effectiveness:** It refers to the all ability of the test generator to either successfully create a test for a fault or to definitively prove its non-testability.